
Conditioning Tree-Based Diffusions and Flows for Probabilistic Tabular Regression

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We extend Treeffuser, a score-based diffusion model for probabilistic tabular
2 regression with gradient-boosted trees, by treating regression conditioning as the
3 central design axis. On the score side, conditional-mean residualization, EDM-
4 style preconditioning, log-sigma time sampling, and noise-level features make
5 the denoising regression better matched to histogram-split tree learners. On the
6 flow side, Gaussian-path flow matching gives a directly learned velocity field
7 with algebraic score recovery, enabling both few-step deterministic ODE sampling
8 and stochastic-interpolant diagnostics. Across ten UCI benchmarks with fold-0
9 tuning and folds-1–5 evaluation, the score-side recipe gives the best Treeffuser
10 calibration, reducing 95% coverage error from 0.023 to 0.015 and cutting sampling
11 time $6.5\times$ over the published baseline. The FM row gives the best normalized
12 CRPS among Treeffuser rows and is another $2.4\times$ faster than the improved score
13 model, but gives back some coverage. These are Pareto improvements rather than
14 decisive dominance: gains over the published baseline are directional rather than
15 conventionally significant ($p \approx 0.07$), and tuned non-diffusion baselines still win
16 several datasets on raw CRPS. Stochastic ($\varepsilon > 0$) samplers do not Pareto-dominate
17 the deterministic corner, so the empirical recommendation is $\varepsilon = 0$ when aggregate
18 CRPS and latency are the target, while stochasticity remains a conditional-tail
19 calibration knob.

20 1 Introduction

21 Probabilistic regression on tabular data must combine the predictive strength of gradient-boosted
22 trees with full predictive distributions usable for calibration, decision-making, and uncertainty
23 quantification. Treeffuser [Beltrán-Velez et al., 2024] achieves this by training LightGBM [Ke et al.,
24 2017] regressors to estimate the score of a noise-perturbed conditional density and integrating the
25 corresponding reverse-time stochastic differential equation (SDE) [Song et al., 2021] at inference. The
26 framework inherits the calibration properties of score-based diffusion while avoiding neural-network
27 density estimators.

28 Our notion of tractability is computational rather than likelihood-exact: predictive quantities are
29 Monte Carlo estimates, but training reduces to supervised tree regression, the final FM row samples
30 with a five-step Heun ODE, and Eq. (2) recovers scores when stochastic sampling is desired.

31 We develop Treeffuser along two axes: score-side modifications matched to histogram-bin tree
32 learners, and Gaussian-path flow matching [Lipman et al., 2023, Liu et al., 2023], which removes the
33 indirect score-to-drift mapping and opens access to stochastic interpolants [Albergo et al., 2023] with
34 non-negative stochasticity schedule $\varepsilon(t)$.

35 **Contributions.** We make three contributions:

1. *Score-side modifications for tree-based diffusion*: conditional-mean residualization, EDM preconditioning, log- σ time sampling, and an explicit log- σ feature for LightGBM score regression (§2.1).
2. *Flow matching for Treeffuser*: a Gaussian-path FM trainer with score recovery when stochastic-interpolant sampling or endpoint-error analysis requires it (§2.2, Lemma 1).
3. *Regression-conditioning unification*: regression conditioning for the tree ensemble as the bottleneck behind both score- and flow-side gains (§4, Appendix A).

Across ten UCI benchmarks, with one fold for tuning and five for evaluation, the score-side recipe gives the best aggregate calibration and CRPS skill score among Treeffuser rows, while FM gives the best aggregate normalized CRPS and fastest Treeffuser sampler. The shared lesson is that noising path, parameterization, training distribution, features, and sampler must make the induced supervised problem well matched to histogram splits.

2 Method

2.1 Score-Side Modifications

The published Treeffuser uses VESDE marginals with uniform t sampling, a noise-prediction target, and LightGBM features $[y_t, X, t]$. We change four pieces: response/input conditioning, allocation of training rows over noise levels, explicit noise-level features, and sampler efficiency.

Conditional-mean residualization. We fit a cross-validated LightGBM mean predictor $\hat{\mu}(x)$ and apply diffusion to $y - \hat{\mu}(x)$ rather than y , moving conditional location out of the score-regression target. The residualizer uses a high-capacity configuration selected from a prior diagnostic sweep (Appendix D) and then fixed; it is not tuned jointly with the diffusion or flow hyperparameters. On large datasets this fixed residualizer can become a bottleneck once the inner model has enough data to represent the conditional mean directly. Residualization is not a minor knob: on the few-step FM ODE it improves aggregate CRPS and 90% coverage error decisively (81/100 paired wins on |cE|@90; Appendix D.1), while the five-capacity A–E residualizer sweep spans CRPSS 0.509–0.523 and 95% tail coverage error 0.010–0.021. The headline C configuration and higher-capacity E are close on aggregate CRPSS (0.523 vs. 0.522), but E improves 95% tail coverage error (0.010 vs. 0.011) and overtakes C on protein in a separate full-data diagnostic (Appendix D.2). This is why we fix residualizer-C rather than tune it jointly, and why automatic, dataset-adaptive residualizer selection is the most likely route to closing the large-dataset gap.

EDM preconditioning. Following Karras et al. [2022], we predict a denoised target $D_\theta(y_t, t)$ scaled to unit variance across noise levels and reconstruct the score $s(y_t, t) = (D_\theta - y_t)/\sigma(t)^2$. This stabilizes the regression target for trees, which would otherwise need to extrapolate noise-prediction magnitudes across t .

Log-sigma time sampling. We draw $\log \sigma(t) \sim \mathcal{N}(-1.2, 1.2^2)$, clip to the SDE’s achievable range, and invert. Because trees bin features by data density, the training-time distribution directly controls where the score model spends capacity.

Log-sigma noise feature. The feature vector for LightGBM becomes $[y_t, X, t, \log \sigma(t)]$, giving the regressor an explicit noise-level signal aligned with EDM scaling.

2.2 Flow Matching with Gaussian Paths

A Gaussian flow path interpolates data y_0 and a standard-normal prior z via

$$y_t = \alpha(t) y_0 + \beta(t) z, \quad t \in [0, 1], \quad (1)$$

with boundary conditions $\alpha(0) = 1, \alpha(1) = 0, \beta(0) = 0, \beta(1) = 1$. We compare three paths: *linear* ($\alpha = 1 - t, \beta = t$), *trigonometric* ($\alpha = \cos(\pi t/2), \beta = \sin(\pi t/2)$), and a *VP* schedule with linear β_t giving $\alpha^2(t) = \exp(-T(t))$, $T(t) = \frac{1}{2}\beta_{\min}t + \frac{1}{4}(\beta_{\max} - \beta_{\min})t^2$. The training target is the conditional velocity $u_t = \alpha'(t)y_0 + \beta'(t)z$, regressed against the model’s $v_\theta(y_t, x, t)$.

81 **Velocity-to-score formula.** For any Gaussian path with Wronskian $W(t) = \alpha(t)\beta'(t) - \alpha'(t)\beta(t)$,
 82 the conditional score is algebraically recoverable from the population velocity:

$$\nabla \log p_t(y_t | x) = \frac{\alpha'(t) y_t - \alpha(t) v_\theta(y_t, x, t)}{W(t) \beta(t)}. \quad (2)$$

83 For linear FM this reduces to $-(y_t + (1 - t)v)/t$; for trig to $-y_t - (2/\pi) \cot(\pi t/2)v$. We use
 84 this identity operationally: it lets a velocity-trained Treeffuser enter the same stochastic-interpolant
 85 sampler family as score matching and exposes the endpoint amplification studied in Appendix A.

86 **Stochastic-interpolant family.** Albergo et al. [2023] show that any non-negative $\varepsilon(t)$ defines a
 87 marginal-preserving reverse-time sampler at the population velocity/score:

$$dy_\tau = \left(-v_\theta + \frac{\varepsilon^2}{2} s_\theta\right) d\tau + \varepsilon dW_\tau, \quad (3)$$

88 where $\tau = 1 - t$ is reverse time and s_θ is the score from (2). The choice $\varepsilon \equiv 0$ recovers the
 89 deterministic probability-flow ODE; positive $\varepsilon(t)$ broadens samples, with exact marginal preservation
 90 only for the true velocity/score. We use the schedule $\varepsilon(t) = ct$, which vanishes at the data endpoint
 91 and suppresses recovered-score error in the stochastic drift (see Proposition 2).

92 **Why these choices suit trees.** Eq. (2) makes FM compatible with stochastic-interpolant samplers;
 93 variance-preserving paths keep the tree input y_t near unit scale, unlike a linear path with $\text{Var}(y_t) \approx$
 94 $(1 - t)^2 + t^2$; and $\varepsilon = 0$ avoids recovered-score error in the stochastic drift (Appendix A). The path
 95 choice is therefore a finite-regression and sampler tradeoff, not a theorem that VP must dominate
 96 every tuned linear-flow row.

97 2.3 Implementation Notes

98 The FM code reuses Treeffuser’s per-output-dimension LightGBM fitter and feature pipeline; only the
 99 target and prior change. Sampling uses the same Heun integrator and residualizer as the score path,
 100 and both training and sampling avoid the singular endpoint with $t \geq 10^{-5}$. Because the learned score
 101 or velocity components are ordinary LightGBM regressors, native LightGBM handling of categorical
 102 features, missing-value splits, and constraints can be used through the same feature pipeline and
 103 parameter pass-through.

104 3 Experiments

105 **Datasets and protocol.** Ten UCI benchmarks (Table 3) are evaluated with a nested tuning protocol.
 106 We split each dataset into six folds: fold 0 is used only for hyperparameter selection, and folds 1–5
 107 are evaluation folds. For each dataset/model family, Optuna tunes on the fold-0 train/validation split;
 108 the selected configuration is then refit and evaluated on each held-out evaluation fold. Evaluation-fold
 109 training rows range from 256 on yacht to 38,108 on protein. For sample-based methods, each test
 110 point uses 200 samples; Treeffuser sampler choices are detailed in Appendix B. Each predictive
 111 metric is therefore a Monte Carlo estimate, but it is averaged over all held-out test points and five
 112 evaluation folds, so the residual sampling error in the aggregate CRPS and coverage numbers is small
 113 relative to the dataset-level standard errors reported in Table 4. Metrics are averaged over evaluation
 114 folds and, for headline summaries, over datasets.

115 **Metrics.** We report CRPS skill score $\text{CRPSS} = 1 - \text{CRPS}_{\text{model}}/\text{CRPS}_{\text{climatology}}$, mean rel-CRPS,
 116 DSS [Dawid, 1984], PIT-KS pass rate at $\alpha = 0.05$, absolute coverage error at 50/90/95% nominal
 117 levels [Gneiting and Raftery, 2007], and sample-generation time. CRPSS and rel-CRPS are the
 118 aggregate comparison metrics because raw CRPS is target-scale dependent; mean climatological
 119 CRPS ranges from 0.0043 on naval to 44.8 on diabetes. Timings are measured on one Apple-
 120 arm64 workstation, exclude fitting, and include generating 200 samples per test point plus inverse
 121 transformations. They use practical benchmark settings rather than a single-thread cap: Treeffuser
 122 samples in batches with `n_parallel=20` and LightGBM `n_jobs=-1`; other libraries use their
 123 configured parallel defaults.

Table 1: Headline real-data results, mean over 10 UCI benchmarks with fold 0 used for tuning and folds 1–5 used for evaluation. rel-CRPS is the per-dataset CRPS divided by the best displayed variant on that dataset. $|cE|@q$ is absolute coverage error at the $q\%$ interval level. The CARD row is a compact benchmark adapter, not a fully optimized CARD pipeline (see Appendix H).

Variant	CRPSS $\uparrow$	rel-CRPS $\downarrow$	KS $p > .05 \uparrow$	$ cE @50 \downarrow$	$ cE @90 \downarrow$	$ cE @95 \downarrow$	samp. time
Treefuser-published	0.670	1.242	0.40	0.117	0.044	0.023	221.35 s
Treefuser-score ⁺ (ours)	0.681	1.131	0.62	0.069	0.020	0.015	34.12 s
Treefuser-FM (ours)	0.680	1.117	0.46	0.064	0.028	0.024	14.41 s
QReg-LightGBM	0.668	1.459	0.34	0.066	0.041	0.044	0.30 s
CatBoost-unc.	0.638	1.409	0.28	0.093	0.078	0.066	0.01 s
NGBoost	0.601	1.550	0.16	0.119	0.114	0.101	0.04 s
Deep ensemble	0.654	1.519	0.26	0.090	0.032	0.018	0.03 s
iBUG	0.649	1.471	0.36	0.111	0.051	0.065	2.89 s
CARD-style diffusion	0.631	1.632	0.24	0.086	0.057	0.043	42.84 s

Variants compared. Three Treefuser rows compare bundled recipes: (1) PUBLISHED—the original baseline with noise-prediction, uniform t , no residualization; (2) SCORE+—the baseline plus §2.1; (3) FM—VP flow matching with residualization and a 5-step Heun ODE. Each row is tuned per dataset under the same fold-0 protocol. The residualizer-C configuration used in the headline rows was selected from the prior diagnostic sweep in Appendix D and then fixed; it is not tuned jointly with the diffusion or flow hyperparameters. We also compare against six per-dataset tuned probabilistic baselines: NGBoost [Duan et al., 2020], quantile-regression LightGBM [Ke et al., 2017], CatBoost RMSE-with-uncertainty [Prokhorenkova et al., 2018], iBUG [Brophy and Lowd, 2022], a Gaussian deep ensemble [Lakshminarayanan et al., 2017], and CARD-style neural diffusion [Han et al., 2022].

3.1 Headline Result

Table 1 reports cross-dataset means under the tuning/evaluation protocol. The score-side recipe lifts CRPSS from 0.670 to 0.681, raises PIT-uniformity pass rate from 0.40 to 0.62, reduces 90/95% coverage error from 0.044/0.023 to 0.020/0.015, and cuts Treefuser sample time $6.5\times$. FM is close on CRPSS, has the best aggregate normalized CRPS among displayed rows, and is another $2.4\times$ faster than score⁺, but gives back part of score⁺’s coverage advantage. The headline FM row uses the VP path because it is the fastest Treefuser operating point; the tuned mechanism ablation identifies residualized linear FM as the CRPS-optimal FM corner, so we treat VP and linear as latency and CRPS endpoints rather than a single winning path (Table 5). Per-dataset raw-CRPS winners span five families; a Treefuser variant wins on 6/10 datasets, with the remaining wins split across deep ensembles, CARD-style diffusion, and quantile-regression LightGBM. The Treefuser aggregate advantage is directional rather than conventionally significant: paired Wilcoxon tests give one-sided $p = 0.080$ for score⁺ over published and $p = 0.065$ for FM over published; FM versus score⁺ remains indistinguishable ($p = 0.922$, two-sided). Because the external families are tuned on an equal finite-trial budget but with differing search spaces and per-trial costs (§4), the cross-family rows are informative about the tradeoff surface rather than definitive head-to-head rankings.

End-to-end timing. Table 1 excludes fitting, but fit time is logged on the same runs, so Table 2 reports both. Two points follow. First, FM is the cheapest end-to-end Treefuser row (28.4 s fit+sample), ahead of score⁺ (46.7 s) and far ahead of the published SDE (237.9 s), so the sampling speedup is not bought back at fit time. Second, near-zero sampling time does not imply lower end-to-end cost: quantile-regression LightGBM samples in 0.30 s but fits 24–49 separate quantile models, so its total cost is 69.6 s, and the deep ensemble is close to score⁺ at 46.3 s but still slower than FM. Fit times are still environment-specific and use library-default parallelism, so we read them as the same kind of practical engineering benchmark as the sampling timings.

3.2 Per-Dataset CRPSS

Table 3 shows the Treefuser per-dataset CRPSS. Score⁺ is best on concrete and diabetes, FM on energy, kin8nm, naval, power, and yacht, and the published baseline on wine and the two largest datasets. Thus the tuned comparison exposes a Pareto surface: score⁺ is usually the calibrated choice, FM the fast normalized-CRPS choice, and the published SDE remains competitive when full SDE sampling is affordable.

Table 2: Fit, sample, and end-to-end time (mean over the ten UCI benchmarks and evaluation folds; same runs as Table 1). Fit time is per evaluation fold; sample time generates 200 samples per test point.

Variant	fit (s)	sample (s)	total (s)
Treeffuser-published	16.57	221.35	237.92
Treeffuser-score ⁺ (ours)	12.54	34.12	46.66
Treeffuser-FM (ours)	14.00	14.41	28.41
QReg-LightGBM	69.34	0.30	69.65
CatBoost-unc.	2.34	0.01	2.35
NGBoost	7.83	0.04	7.87
Deep ensemble	46.26	0.03	46.29
iBUG	0.98	2.89	3.87
CARD-style diffusion	23.19	42.84	66.03

Table 3: Per-dataset CRPSS (higher is better) and post-split training rows.

Dataset	Train rows	Published	Score ⁺	FM
yacht	256	0.941	0.962	0.962
diabetes	368	0.201	0.241	0.226
energy	640	0.956	0.962	0.964
concrete	858	0.740	0.774	0.771
wine	5414	0.346	0.305	0.307
kin8nm	6826	0.555	0.617	0.631
power	7973	0.829	0.839	0.843
naval	9945	0.950	0.949	0.951
cali. housing	17200	0.687	0.676	0.674
protein	38108	0.494	0.480	0.472

Table 4: Robustness summary from the tuned evaluation artifacts. Ranks average per-dataset ranks after first averaging folds 1–5; lower ranks are better. SE is the standard error over the ten dataset-level CRPSS means.

Model	CRPS rank	CRPSS $\pm$ SE	rel-CRPS	cE @95 rank
Treeffuser-FM	2.80	0.680 $\pm$ 0.086	1.117	4.00
Treeffuser-score ⁺	3.00	0.681 $\pm$ 0.085	1.131	2.10
QReg-LightGBM	4.00	0.668 $\pm$ 0.079	1.459	5.60
Treeffuser-published	4.10	0.670 $\pm$ 0.084	1.242	3.30
Deep ensemble	5.80	0.654 $\pm$ 0.082	1.519	2.90
iBUG	5.80	0.649 $\pm$ 0.081	1.471	6.50
CatBoost-unc.	6.00	0.638 $\pm$ 0.088	1.409	7.10
CARD-style diffusion	6.60	0.631 $\pm$ 0.091	1.632	5.70
NGBoost	6.90	0.601 $\pm$ 0.094	1.550	7.80

3.3 Robustness Checks

Table 4 summarizes the same tuned artifacts by per-dataset ranks and dataset-level standard errors. The Treeffuser rows occupy the two best mean CRPS ranks, but the aggregate gaps are small relative to the dataset standard errors, and the rank tests do not support a one-family sweep. A Friedman test over the nine displayed families rejects equal CRPS ranks ($p = 0.001$), but the Nemenyi critical difference is 3.80 mean-rank points; post-hoc gaps separate FM and score⁺ only from the weakest rows, not from the published SDE, quantile-regression LightGBM, or deep ensembles. This supports the intended reading of Table 1: the aggregate Treeffuser gains are robustly directional, while individual datasets and metrics still have mixed winners.

3.4 Mechanism Ablation

Table 5 isolates the main design choices under the same fold-0 tuning protocol and the same LightGBM hyperparameter search surface. Three conclusions matter for the mechanism story. First, changing only the published noise-prediction score model from Euler SDE sampling to a 25-step Heun probability-flow ODE is not a free speedup: CRPSS falls from 0.670 to 0.629. Second, EDM input/target preconditioning is the largest score-side jump, improving CRPSS from 0.646 after residualization alone to 0.680 and cutting 95% coverage error from 0.027 to 0.016. The log- σ feature

Table 5: Tuned mechanism ablation over the same ten UCI datasets. Each row uses the same LightGBM hyperparameter search surface and fold-0 tuning / folds-1–5 evaluation protocol; fixed method choices and bound samplers differ by row. rel-CRPS is normalized by the best ablation row on each dataset.

Variant	CRPSS $\uparrow$	rel-CRPS $\downarrow$	q -MACE $\downarrow$	cE @90 $\downarrow$	cE @95 $\downarrow$	samp. time	raw wins
Score noise, Euler-50	0.670	1.150	0.044	0.044	0.023	217.30 s	2
Score noise, Heun-25	0.629	1.424	0.058	0.049	0.025	190.92 s	0
+ residualizer	0.646	1.104	0.041	0.044	0.027	47.58 s	0
+ EDM input/target	0.680	1.042	0.034	0.025	0.016	109.46 s	0
+ log- σ feature	0.679	1.042	0.035	0.027	0.017	73.10 s	0
Score ⁺ full	0.681	1.045	0.028	0.020	0.015	35.11 s	0
FM linear + residualizer	0.684	1.027	0.028	0.024	0.020	19.28 s	4
FM trig + residualizer	0.681	1.027	0.028	0.026	0.021	23.87 s	2
FM VP, no residualizer	0.631	1.985	0.050	0.051	0.032	23.19 s	1
FM VP + residualizer	0.680	1.034	0.034	0.028	0.024	14.69 s	1

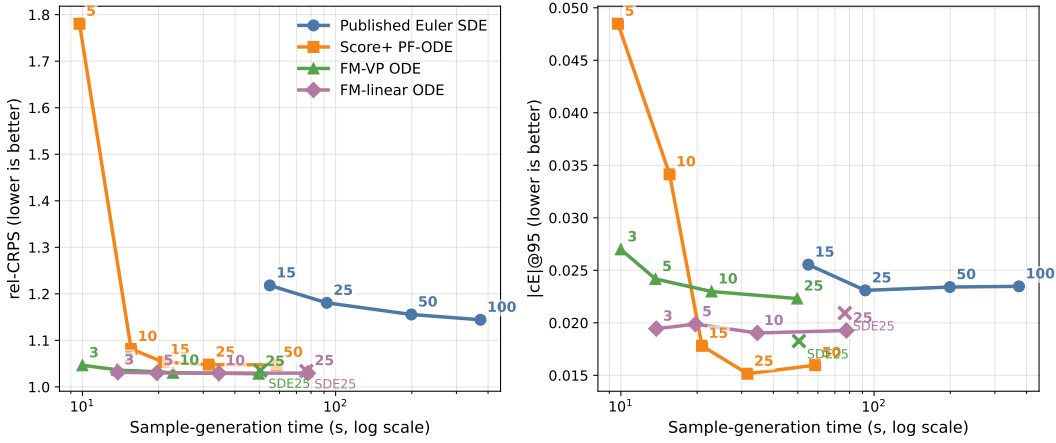


Figure 1: Sampler cost tradeoff after tuning, averaged over the same ten UCI datasets and folds 1–5. Points are eval-only sampler changes from the tuned configurations; labels show solver steps, and SDE25 marks the 25-step stochastic FM sampler. rel-CRPS is normalized by the best sampler-cost row on each dataset.

is roughly neutral under uniform t , while the full score⁺ bundle is the most calibrated score-side row. Third, few-step FM needs residualization: VP-FM without residualization is the weakest rel-CRPS row despite one small-dataset win. Among residualized FM paths, tuned linear has the best aggregate CRPSS, trig is close and wins two datasets, and VP is the fastest. This weakens a simple “VP geometry always wins” reading, but supports the broader finite-regression view: feature scale, velocity target shape, residualization, and ODE discretization all affect the supervised problem solved by LightGBM.

3.5 Sampler Cost Tradeoff

Figure 1 re-evaluates the tuned Treeffuser configurations across sampler step counts without retuning hyperparameters. The practical pattern is sharp. FM-ODE is effectively converged by 3–5 Heun steps: VP-FM moves from rel-CRPS 1.046 at 3 steps to 1.037 at 5 steps, and residualized linear FM is already at 1.031 at 3 steps. Score⁺ needs more integration: 5-step PF-ODE is not competitive (rel-CRPS 1.781), while 15–25 steps recover the calibrated score row. The published Euler SDE improves gradually with step count but remains much slower at comparable aggregate CRPS. Finally, the 25-step FM-SDE points confirm the calibration tradeoff: for VP-FM, stochasticity improves 95% coverage error from 0.024 to 0.018, but worsens q -MACE and 90% coverage error while paying 25 SDE steps. The same tradeoff is clearest in the highest predicted-IQR bin: Appendix K shows VP-FM 95% coverage improving from 0.906 with the 5-step ODE to 0.944 with the 25-step SDE, and linear FM from 0.902 to 0.935. Thus the default recommendation remains few-step FM-ODE for aggregate CRPS/latency, with stochasticity reserved for conditional-calibration targets.

Design diagnostics. Fixed diagnostics explain why particular bundles were tested, but they are not separately tuned factorial comparisons. For FM, the fixed path diagnostic in Appendix G isolates the expected feature-scale advantage of variance-preserving paths under matched residualizer-C and a 5-step Heun ODE. The tuned ablation in Table 5 is the performance evidence: it shows a close residualized path frontier, with linear strongest on aggregate CRPSS, trig competitive and winning two datasets, and VP fastest. Thus we treat path choice as a practical conditioning/sampling tradeoff rather than a settled hierarchy. Stochastic samplers do not Pareto-dominate $\varepsilon = 0$ on aggregate CRPS/coverage/latency, though they can improve high-IQR tail coverage (Appendix K); transferred score-side tricks are neutral or worse. This is why the recommendation is deterministic ODE for aggregate CRPS/latency, with stochasticity left as a conditional-calibration knob. Appendix C maps tuned headline claims versus fixed diagnostics.

4 Discussion

Two findings deserve emphasis. First, the headline rows compare tuned bundled recipes, while Appendix C scopes the fixed diagnostics. Because score^+ and FM share the same LightGBM tuning surface, the FM result is not implicitly absorbing tuning-budget improvements that belong to the score track. Second, per-dataset tuning turns the headline into a tradeoff: score^+ is the strongest calibrated Treffuser row, whereas FM is strongest on normalized CRPS and latency. The common mechanism is regression conditioning for tree ensembles: EDM preconditioning stabilises the VE score path’s inputs and denoising residual, while FM changes the target and sampler geometry of the supervised problem. The tuned path ablation is important here: variance preservation is one helpful conditioning axis, but not the whole story. Linear FM has a less uniform input scale, yet its simple displacement-like velocity target and direct few-step ODE can be easier to exploit once conditional location is residualized; trig shows that the variance-preserving family remains competitive, but does not uniquely determine the CRPS optimum. Log- σ sampling controls where tree capacity is spent, the log- σ feature exposes the noise regime to histogram splits, residualization removes predictable conditional location, and Heun ODE sampling is an inference-side efficiency gain. The large-dataset inversion argues against a blanket “SDE is worse” interpretation: per-IQR-bin diagnostics show that stochasticity can improve high-uncertainty tail coverage, while aggregate metrics penalise broader intervals, worse CRPS, and higher latency.

Limitations. All displayed families are tuned per dataset, but this is still finite-budget model selection with unequal search spaces and per-trial costs: the comparison is equal finite-trial budget rather than wall-clock-normalized model selection. We include six probabilistic baselines but not distributional random forests [Cevic et al., 2022]; per-dataset gaps, especially on large datasets, should be read as descriptive rather than high-powered significance estimates. The benchmark suite is also limited to scalar UCI-style regression; categorical-heavy tables, multi-output responses, and operational tail-risk settings remain outside the evidence in this paper. The residualizer is selected from fixed diagnostics rather than automated jointly with the diffusion objective, which is the most likely reason the published SDE still wins the two largest datasets, where the inner model has enough data to fit the conditional mean directly and the fixed residualizer becomes a bottleneck. Finally, the benchmark is scalar, and this matters for the target dimension: the implementation fits one LightGBM score or velocity regressor per response coordinate, but each regressor sees the full noisy response vector, so cross-output dependence can in principle be represented through the learned vector field. How well this works with diagonal Gaussian noise paths, per-coordinate tree targets, and finite-step samplers is outside the evidence here and left to future work.

Choosing a variant. The per-dataset pattern supports benchmark-level operating points rather than a uniform-dominance claim or a simple *calibration vs. speed* split. Use headline 5-step VP-FM when sample latency or aggregate normalized CRPS is the binding constraint; it is the fastest displayed headline corner, while the mechanism ablation shows residualized linear FM as the tuned CRPS corner and trig as a close path check. Prefer score^+ when interval coverage or PIT uniformity is the reported quantity, since it is the strongest calibrated row at the cost of $\sim 2.4\times$ more sampling time. On the largest datasets, where residualizer capacity bottlenecks, the published SDE remains competitive and full SDE sampling is worth its cost. Stochasticity ($\varepsilon > 0$) is a conditional knob, not a default: select it only when high-uncertainty tail coverage is the target (Appendix K), against conditional-calibration metrics rather than aggregate CRPS.

Related uncertainty methods for trees. Our baselines cover sample-based and quantile tree learners, but several complementary tree-uncertainty families target different quantities. Epistemic-ensemble boosting such as SGLB [Ustimenko and Prokhorenkova, 2021] uses Langevin dynamics to quantify model uncertainty and is evaluated mainly on NLL/RMSE and out-of-distribution detection rather than the sharpness-and-coverage CRPS regime here. Frequentist inference for boosted trees, e.g. Boulevard [Zhou and Hooker, 2022], yields asymptotic prediction intervals with coverage guarantees but does not provide a full generative predictive law. Conformal wrappers such as conformalized quantile regression [Romano et al., 2019] are orthogonal routes to finite-sample marginal coverage; a useful next comparison is to calibrate both Treefuser variants and quantile-regression LightGBM under a matched calibration split and report the resulting coverage–width or interval-score tradeoff.

Future work. Useful extensions include wall-clock-normalized tuning, matched post-hoc recalibration baselines and calibration–width metrics, selection criteria for stochastic samplers when conditional coverage matters, categorical-heavy stress tests, multi-output regression, and automatic residualizer selection.

References

- Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv:2303.08797*, 2023.
- Nicolas Beltrán-Velez, Alessandro Antonio Grant, Alexander Feder, Cinjon Resnick, Kyunghyun Cho, and David M. Blei. Treefuser: Probabilistic predictions via conditional diffusions with gradient-boosted trees. In *Advances in Neural Information Processing Systems*, 2024.
- Jonathan Brophy and Daniel Lowd. Instance-based uncertainty estimation for gradient-boosted regression trees. In *Advances in Neural Information Processing Systems*, 2022.
- Domagoj Cevic, Loris Michel, Jeffrey Naef, Nicolai Meinshausen, and Peter Bühlmann. Distributional random forests: Heterogeneity adjustment and multivariate distributional regression. *Journal of Machine Learning Research*, 23(333):1–79, 2022.
- A. Philip Dawid. Statistical theory: The prequential approach. *Journal of the Royal Statistical Society. Series A*, 147(2):278–292, 1984.
- Tony Duan, Anand Avati, Daisy Yi Ding, Khanh K. Thai, Sanjay Basu, Andrew Y. Ng, and Alejandro Schuler. NGBoost: Natural gradient boosting for probabilistic prediction. In *International Conference on Machine Learning*, 2020.
- Tilman Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.
- Xizewen Han, Huangjie Zheng, and Mingyuan Zhou. CARD: Classification and regression diffusion models. In *Advances in Neural Information Processing Systems*, 2022.
- Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems*, 2022.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 2017.
- Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, 2017.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.
- Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.

- 299 Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey
300 Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information*
301 *Processing Systems*, 2018.
- 302 Yaniv Romano, Evan Patterson, and Emmanuel J. Candès. Conformalized quantile regression. In
303 *Advances in Neural Information Processing Systems*, 2019.
- 304 Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben
305 Poole. Score-based generative modeling through stochastic differential equations. In *International*
306 *Conference on Learning Representations*, 2021.
- 307 Aleksei Ustimenko and Liudmila Prokhorenkova. SGLB: Stochastic gradient langevin boosting. In
308 *International Conference on Machine Learning*, 2021.
- 309 Yichen Zhou and Giles Hooker. Boulevard: Regularized stochastic gradient boosted trees and their
310 limiting distribution. *Journal of Machine Learning Research*, 23(183):1–44, 2022.

311 A Theoretical Derivations and Design Consequences

312 **Lemma 1 (Wronskian velocity-to-score identity)** *For a differentiable Gaussian path $y_t =$*
313 *$\alpha(t)y_0 + \beta(t)z$ with conditional velocity $u_t = \alpha'(t)y_0 + \beta'(t)z$, nonzero Wronskian $W(t) =$*
314 *$\alpha(t)\beta'(t) - \alpha'(t)\beta(t)$, and $\beta(t) \neq 0$ (away from the data endpoint), the population velocity*
315 *$v(y_t, x, t) = \mathbb{E}[u_t \mid y_t, x, t]$ implies the conditional score*

$$\nabla \log p_t(y_t \mid x) = \frac{\alpha'(t)y_t - \alpha(t)v(y_t, x, t)}{W(t)\beta(t)}.$$

316 *Proof.* Solving the two linear equations for z gives

$$z = \frac{\alpha(t)u_t - \alpha'(t)y_t}{\alpha(t)\beta'(t) - \alpha'(t)\beta(t)} = \frac{\alpha(t)u_t - \alpha'(t)y_t}{W(t)}.$$

317 Because $p(y_t \mid y_0, x) = \mathcal{N}(\alpha y_0, \beta^2 I)$, the conditional score is $-z/\beta(t)$. Fisher’s identity gives
318 $\nabla \log p_t(y_t \mid x) = \mathbb{E}[-z/\beta(t) \mid y_t, x, t]$. Substituting the expression for z and using that y_t is fixed
319 under the conditional expectation yields the claim. Replacing v by v_θ gives Eq. (2). $\square$

320 The identity is not new theory: it specializes the standard velocity/score relations for Gaussian
321 probability paths and stochastic interpolants [Lipman et al., 2023, Albergo et al., 2023] to the
322 Wronskian form we use, and we present it only as the bridge that lets a velocity-trained tree model
323 drive score-based and stochastic-interpolant samplers without retraining.

324 At a fixed t with $\alpha(t) \neq 0$, the same identity inverts to $v(y_t, x, t) = \alpha'(t)y_t/\alpha(t) -$
325 $W(t)\beta(t)\nabla \log p_t(y_t \mid x)/\alpha(t)$. Thus score and velocity have the same conditional structure as
326 functions of y_t at that time; the practical differences studied here come from path-dependent scale
327 variation across t , feature geometry, and sampler dynamics rather than an intrinsic fixed- t smoothness
328 advantage.

329 **Path-induced feature scale.** If y_0 and z are independent with unit variance after standardization,
330 then the path feature satisfies

$$\text{Var}(y_t) = \alpha(t)^2 + \beta(t)^2.$$

331 Thus the linear path has $\text{Var}(y_t) = (1-t)^2 + t^2$ and compresses to variance $1/2$ at $t = 1/2$, whereas
332 the trigonometric and VP paths keep $\text{Var}(y_t) = 1$ for all t . This identity is elementary; its role here
333 is to connect path choice to the empirical distribution from which histogram split candidates are built.

334 **Proposition 1 (Pooled histogram-bin proxy)** *Fix a scalar noised-response feature and a finite par-*
335 *tition $\mathcal{P} = \{I_1, \dots, I_B\}$. Let $\mathcal{H}(\mathcal{P})$ be the functions that are constant on each I_b , and let*

$$A_t(\mathcal{P}) = \inf_{g \in \mathcal{H}(\mathcal{P})} \mathbb{E}_{p_t}[(f_t(y_t) - g(y_t))^2], \quad f_t(y) = \mathbb{E}[T_t \mid y_t = y, x, t],$$

where T_t is the scalar score, denoising, noise, or velocity target at time t with covariate value x fixed.
Then

$$A_t(\mathcal{P}) \leq \sum_{b=1}^B p_t(I_b) \text{osc}_{I_b}(f_t)^2,$$

where $\text{osc}_{I_b}(f_t) = \sup_{y, y' \in I_b} |f_t(y) - f_t(y')|$. If f_t is L_t -Lipschitz on the bins, then

$$A_t(\mathcal{P}) \leq L_t^2 \sum_{b=1}^B p_t(I_b) |I_b|^2.$$

Moreover, in a compact-support proxy with smooth positive density q and equal- q -mass bins $\mathcal{P}_q$, the high-resolution scaling is

$$A_t(\mathcal{P}_q) \lesssim \frac{L_t^2}{B^2} \int \frac{p_t(y)}{q(y)^2} dy,$$

up to lower-order bin-width terms.

Proof. The best piecewise-constant approximation on a fixed partition cannot be worse than any particular constant choice on each bin. Choosing any value between the infimum and supremum of f_t on I_b gives pointwise squared error at most $\text{osc}_{I_b}(f_t)^2$ on that bin, and integration under p_t gives the first bound. The Lipschitz bound follows from $\text{osc}_{I_b}(f_t) \leq L_t |I_b|$. For the high-resolution proxy, an equal- q -mass bin has $|I_b| \approx 1/(Bq(\xi_b))$ for some $\xi_b \in I_b$, so

$$\sum_b p_t(I_b) |I_b|^2 \approx \sum_b p_t(\xi_b) |I_b|^3 = \frac{1}{B^2} \sum_b \frac{p_t(\xi_b)}{q(\xi_b)^3} \frac{1}{B} \rightarrow \frac{1}{B^2} \int \frac{p_t(y)}{q(y)^2} dy,$$

where the last step is a Riemann sum under the measure with density q . $\square$

Shared-bin mismatch. Proposition 1 is a one-feature proxy for the finite split resolution available to a histogram tree, not a theory of the full boosted LightGBM ensemble. In the benchmark, rows from many noise levels are pooled before tree fitting, so a natural proxy is $q = \bar{p}$ with

$$\bar{p}(y) = \int p(t) p_t(y) dt.$$

The high-resolution term $\int p_t / \bar{p}^2$ is large when $\bar{p}$ allocates little bin density to regions where a particular p_t still places mass. VE paths create exactly this compromise by mixing data-scale low-noise features with expanded high-noise features; a single pooled histogram must spend resolution across both regimes. EDM input preconditioning and VP-FM both reduce this mismatch by keeping the tree feature scale closer to a common range across t . Figure 2(b) illustrates the same effect empirically: the published-style VE target is stable, but the unpreconditioned tree input expands across noise levels, whereas EDM and VP-FM keep the feature scale closer to a shared-bin regime. This is a finite-bin conditioning statement, not a claim that the fixed-time VE regression function is intrinsically rough.

Residualization removes predictable location. Residualization is an exact reparameterization in the infinite-capacity limit: for any fixed $\hat{\mu}(x)$, modelling $r = y - \hat{\mu}(x)$ exactly and returning $y = r + \hat{\mu}(x)$ recovers the same conditional distribution. Its empirical value is finite-model conditioning. In the scalar case, if $\mu(X) = \mathbb{E}[Y | X]$ and $R = Y - \mu(X)$, the law of total variance gives

$$\text{Var}(Y) = \mathbb{E}[\text{Var}(Y | X)] + \text{Var}(\mu(X)), \quad \text{Var}(R) = \mathbb{E}[\text{Var}(Y | X)].$$

For vector-valued responses the same statement applies to the trace of the covariance. Thus residualization removes predictable location variation before noised responses are pooled into histogram bins. The large-dataset caveat remains: a fixed residualizer can become a bottleneck once the inner score or velocity model has enough data to represent the conditional mean directly.

Proposition 2 (Endpoint score-error amplification) Let $v_\theta(y_t, x, t) = v(y_t, x, t) + \delta(y_t, x, t)$ be an approximate velocity with pointwise error δ , and let $\hat{s}_\theta, s$ denote the implied and true scores from Lemma 1. Then

$$\hat{s}_\theta(y_t, x, t) - s(y_t, x, t) = -\frac{\alpha(t)}{W(t)\beta(t)} \delta(y_t, x, t),$$

so the pointwise score-error gain is $|\alpha(t)/(W(t)\beta(t))|$. For the linear and trigonometric paths, $\beta(t) \asymp t$ and $W(t)$ is bounded away from zero near $t = 0$, so this gain diverges like $1/t$. For the VP path used here, $\beta(t) \asymp \sqrt{t}$ and $W(t) \asymp 1/\sqrt{t}$, so $W(t)\beta(t)$ is bounded away from zero and the velocity-to-score gain remains bounded.

Proof. Eq. (2) is linear in v , so

$$\hat{s}_\theta - s = \frac{\alpha'(t)y_t - \alpha(t)(v + \delta) - (\alpha'(t)y_t - \alpha(t)v)}{W(t)\beta(t)} = -\frac{\alpha(t)\delta}{W(t)\beta(t)}. \quad \square$$

Endpoint amplification. Proposition 2 therefore predicts that any stochasticity schedule with $\varepsilon(t) \not\rightarrow 0$ as $t \rightarrow 0$ injects recovered-score error into the SDE drift. In Eq. (3), the recovered-score contribution to the drift error is

$$\frac{\varepsilon(t)^2}{2}(\hat{s}_\theta - s) = -\frac{\varepsilon(t)^2\alpha(t)}{2W(t)\beta(t)}\delta.$$

More generally, if $\beta(t) \asymp t^a$, $\varepsilon(t) \asymp t^b$, $\alpha(t) \asymp 1$, and $W(t) \asymp t^{a-1}$ near the data endpoint, this drift-error contribution scales as $t^{2b+1-2a}\delta$. Linear and trigonometric paths have $a = 1$, so constant stochasticity diverges, $\varepsilon(t) \asymp \sqrt{t}$ leaves a non-vanishing endpoint error scale, and $\varepsilon(t) = ct$ makes the recovered-score contribution vanish. The VP path used here has $a = 1/2$, so the velocity-to-score gain is milder, but vanishing stochasticity still suppresses the endpoint contribution and $\varepsilon \equiv 0$ removes it entirely. This is why $\varepsilon(t) = ct$ and the deterministic corner are the two natural schedules in §2.2.

VE population identity. On a VE path $y_t = y_0 + \beta(t)z$ with $\beta'(t) > 0$, normalized velocity, noise prediction, and denoising score recovery imply the same population score:

$$s_t(y_t, x) = -\frac{\mathbb{E}[z \mid y_t, x, t]}{\beta(t)} = \frac{\mathbb{E}[y_0 \mid y_t, x, t] - y_t}{\beta(t)^2}.$$

Indeed, for VE, $\alpha = 1$, $\alpha' = 0$, $u_t = \beta'(t)z$, and $W = \beta'(t)$; Lemma 1 gives $s_t = -\mathbb{E}[z \mid y_t, x, t]/\beta(t)$, and $y_t = y_0 + \beta(t)z$ gives the denoising form. This identity is not a claim that the finite LightGBM problems are identical. EDM fits the preconditioned denoising residual $F = (y_0 - c_{\text{skip}}y_t)/c_{\text{out}}$ and reconstructs $D_\theta = c_{\text{skip}}y_t + c_{\text{out}}F_\theta$, whereas normalized VE-FM would fit z directly. They imply the same score at the optimum but present different targets and feature scalings to LightGBM. This is the useful interpretation of the score⁺-FM tie: both improve the conditioning of the regression problem seen by the tree ensemble, EDM by preconditioning the VE score objective and VP-FM by choosing a better-conditioned probability path.

Table 6: Regression-conditioning view of the main design choices. Here y_0 and z are standardized. “Conditioning issue” refers to the supervised problem fit by LightGBM, not to the population score identity.

Variant	Regressed object	Feature scale	Conditioning issue / fix
VE noise prediction	$-z$	$y_0 + \sigma z$ grows with σ	Unit target, but heteroscedastic input and $1/\sigma$ score amplification.
VE EDM score ⁺	$(y_0 - c_{\text{skip}}y_t)/c_{\text{out}}$	$c_{\text{in}}y_t$ is near unit scale	Input and denoising residual are explicitly preconditioned.
Raw VE-FM	$\beta'(t)z$	$y_0 + \beta z$ grows with β	Velocity target inherits the schedule derivative.
Normalized VE-FM	z	$y_0 + \beta z$ unless also scaled	Population-equivalent to noise prediction after known rescaling.
VP-FM	$\alpha'y_0 + \beta'z$	$\alpha^2 + \beta^2 \approx 1$	Path keeps tree features near the standardized data scale.

B Datasets and Protocol Details

Sources. The ten UCI benchmarks follow the canonical Treeffuser protocol [Beltrán-Velez et al., 2024]: yacht, concrete, energy, wine, kin8nm, naval, power_plant, protein from the UCI repository; california_housing from scikit-learn’s fetch_california_housing;

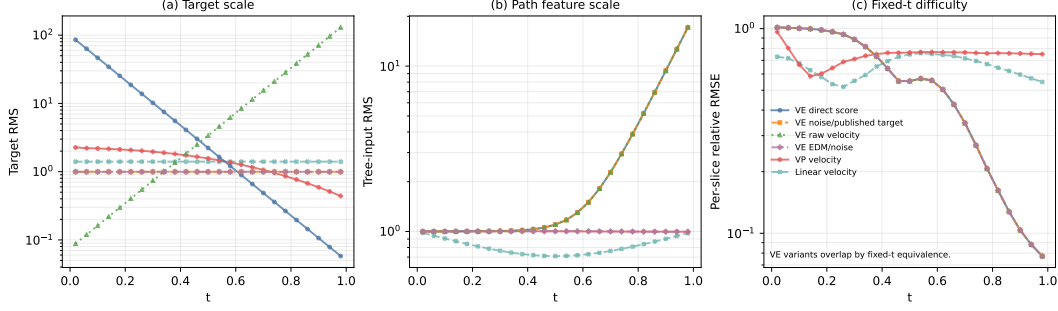


Figure 2: Toy preconditioning diagnostic on a one-dimensional heteroscedastic mixture, generated by `pixi run toy-geometry`. Panel (a) shows that direct VE score prediction and raw VE-FM have target scales that vary by orders of magnitude across t , whereas the published-style VE noise target, EDM/noise prediction, and linear velocity have unit target scale. Panel (b) separates target scaling from input geometry: the published-style VE target is stable but the tree input $y_t = y_0 + \sigma z$ still expands strongly, while EDM rescales the input and VP-FM keeps the path feature near standardized scale. Panel (c) fits a fresh LightGBM at each fixed t ; the VE curves overlap up to numerical noise, confirming that the score/velocity distinction is not intrinsic fixed- t smoothness but cross- t conditioning.

and diabetes from scikit-learn’s `load_diabetes`. The headline results use the six-fold tuning/evaluation protocol from §3; Table 3 reports the post-split training rows for each evaluation fold. Fixed-configuration appendix diagnostics instead use the train/test sizes declared in their YAML configs, sometimes with large datasets subsampled for sweep cost. The only preprocessing is `StandardScaler` standardisation of features and target, fit on the training split and applied to the corresponding test split. `wine` is the union of `wine-quality-red` and `wine-quality-white` with a binary color indicator. Although the implementation can use LightGBM’s native categorical handling, missing-value routing, and constraint options, the paper experiments do not isolate these features or use categorical-heavy benchmark tables.

Seed policy. The fold-tuned headline uses the deterministic six-fold manifest with master seed 0. Fixed diagnostic sweeps use the seed lists declared in their configs (usually 0–2 or 0–9), with three independent streams: `data_seed` (offset 0) for the train/test split; `model_seed` (offset 10,000) for the LightGBM regressors; `sampler_seed` (offset 20,000) for stochastic samplers. Variance reduction across seeds therefore reflects only sampling-and-split variability, not boosting initialisation.

Common training settings. In fixed diagnostic sweeps, unless noted, Treefuser variants use $n_{\text{estimators}} = 3000$, early stopping after 50 rounds, learning rate 0.1, $n_{\text{repeats}} = 30$ (each training point is reused with 30 fresh t samples), and the per-output-dimension LightGBM fitter. In the headline protocol, these and other LightGBM hyperparameters are selected per dataset from the tuning spaces and stored under `benchmarks/results/tuning/best_params/`. Inference draws 200 samples per test point.

Tuning budget. For each dataset/model-family pair, Optuna targets 25 finite trials on the fold-0 train/validation split, with at most twice as many total attempts to allow failed or non-finite trials. The tuning objective is validation CRPS computed from 100 predictive Monte Carlo samples per validation point; this is the number of generated response samples used to estimate CRPS, not a subsample of validation observations. Final evaluation refits the selected configuration on each held-out evaluation fold and draws 200 samples per test point. The three Treefuser rows share the same six-dimensional LightGBM search surface: `n_estimators`, `learning_rate`, `num_leaves`, `max_depth`, `min_child_samples`, and `subsample`; method-defining choices such as objective, residualization, probability path, and sampler are fixed by row. External baselines are also tuned for 25 finite trials, but their search spaces and per-trial costs differ, so the comparison should be read as equal finite-trial budget rather than wall-clock-normalized model selection. The persisted YAML files record `n_trials_target`, `n_trials_finite`, and `n_trials_failed`. Table 7 summarises the per-family tunable surfaces.

Table 7: Tunable hyperparameter ranges per model family (25 Optuna trials each). Log = log-uniform sampling. Fixed method-defining parameters are not listed.

Family	Tunable parameter	Range	Scale
Treeffuser (all 3 rows)	n_estimators	200–3000	log
	learning_rate	0.001–0.3	log
	num_leaves	15–255	linear
	max_depth	{−1, 4, 6, 8, 10}	categorical
	min_child_samples	5–100	linear
	subsample	0.5–1.0	linear
NGBoost	n_estimators	200–3000	log
	learning_rate	0.001–0.3	log
iBUG	k	10–200	log
	n_estimators	200–2000	log
	learning_rate	0.001–0.3	log
	max_depth	3–10	linear
QReg-LightGBM	quantile_count	24–49	linear
	n_estimators	50–500	log
	learning_rate	0.02–0.2	log
	num_leaves	15–127	linear
CatBoost	iterations	200–3000	log
	learning_rate	0.001–0.3	log
	depth	4–10	linear
Deep ensemble	hidden_size	{64, 128, 256, 512}	categorical
	n_layers	2–5	linear
	learning_rate	0.0001–0.01	log
	max_epochs	{100, 200, 400}	categorical
CARD-style	hidden_size	{64, 128, 256}	categorical
	n_layers	2–5	linear
	learning_rate	0.0001–0.01	log
	max_epochs	{200, 400}	categorical
	diffusion_epochs	{200, 400}	categorical
	n_steps	{50, 100, 200}	categorical

Appendix calibration metrics. Appendix tables use q -MACE for sample-rank calibration: for each held-out response, we compute the fraction of predictive samples above the observed value, sort these ranks, and average their absolute deviation from an evenly spaced uniform grid. Lower q -MACE therefore means the sample predictive distributions have ranks closer to uniform, analogous to the PIT-KS metric in the main table but reported as an error magnitude.

Sampler details. TREEFFUSER-PUBLISHED uses the VESDE reverse-time SDE with the Euler–Maruyama sampler at $n_{\text{steps}} = 50$, matching the published code path. TREEFFUSER-SCORE⁺ uses the Heun probability-flow ODE at $n_{\text{steps}} = 25$ steps under the EDM-style parametrization with $\sigma_{\min} = 0.01$, $\sigma_{\max} = 20$. TREEFFUSER-FM uses the Heun ODE at $n_{\text{steps}} = 5$ on the VP path with $\beta_{\min} = 0.1$, $\beta_{\max} = 20$.

C Claim Scope and Diagnostic Cross-References

Unless otherwise stated, appendix sweeps are fixed-configuration diagnostics used to isolate modelling choices; they are not per-dataset tuned headline comparisons. The experimental pipeline is: fixed diagnostics identify plausible design choices; those choices define the frozen PUBLISHED, SCORE⁺, and FM model families; the frozen families are then tuned per dataset under the fold-0 protocol; and only that tuned protocol supports the headline aggregate performance claims.

The four score-side modifications of §2.1 were validated by smoke and synthetic-core sweeps before being combined in the SCORE⁺ variant. The residualizer-C configuration that the headline uses was selected by the residualizer sweep summarised in Appendix D. Log-sigma time sampling, the log-sigma noise feature, and EDM preconditioning were locked in by the log_sigma_sweep, synthetic_core, and pf_ode_sweep development runs. We treat those runs as model-selection evidence rather than as separately reported tuned comparisons; the quantitative claim in the paper is the combined SCORE⁺ row in Table 1, where all four choices are evaluated together on the full ten-dataset protocol.

Table 8: Scope of the empirical claims. “Tuned” means per-dataset Optuna selection on fold 0 before evaluation on folds 1–5; “fixed” means shared diagnostic settings declared in the corresponding YAML configs.

Claim	Evidence	Tuning status	Role in the paper
SCORE ⁺ and FM improve the Treefuser tradeoff surface over the published row	Table 1 and Table 3	Tuned per dataset	Confirmatory performance comparison among frozen model families
SCORE ⁺ is the most calibrated Treefuser row under this benchmark protocol	Table 1	Tuned per dataset	Headline aggregate calibration claim
VP-FM is the preferred FM path among the tested paths	Table 15	Fixed diagnostic	Mechanism and model-selection evidence; not a claim that VP would win every independently tuned path comparison
Deterministic FM is the recommended aggregate CRPS/latency sampler	Table 14 with Table 19	Fixed diagnostic	Sampler diagnostic; stochasticity remains a conditional-calibration knob
Residualizer-C is a reasonable headline residualizer choice	Tables 9 and 10	Fixed diagnostic	Design choice for the frozen model families; automatic residualizer selection is left as future work
Log- σ sampling, explicit noise-level features, and EDM preconditioning motivate the SCORE ⁺ bundle	Toy diagnostic and development sweeps listed in Table 20	Fixed diagnostic	Mechanism evidence for the bundle; the quantitative performance claim is the tuned combined SCORE ⁺ row

D Residualizer Configuration

D.1 Off vs. Mean Residualization (FM)

We isolate the contribution of mean residualization on the FM side at matched configuration (VP path, uniform t , uniform loss weighting, residualizer-C extra parameters) using a ten-dataset mixed synthetic/real diagnostic suite and ten seeds.

Table 9: FM with mean residualization vs. no residualization (off). Paired differences (mean – off, 100 paired observations); negative is better for all metrics shown.

Sampler	Δ CRPS	Δq -MACE	Δ KS stat	$\Delta cE @90$	paired t on Δ CRPS
ODE @ 5 steps	−0.23	−0.022	−0.034	−0.032	−4.30
SDE @ 25 steps	−0.05	+0.001	+0.003	−0.001	−2.91

The ODE-5 advantage is large and broadly significant across calibration metrics (81/100 paired wins on q -MACE and 81/100 on $|cE|@90$). At SDE-25 the gap on calibration metrics is null (paired $t < 1$); the aggregate CRPS gain is driven by two real-data outliers (yacht +54% relative, concrete +15%) while six of ten datasets actually favour the unresidualized baseline. The mechanism is consistent with the few-step ODE being unable to absorb the conditional mean itself when the velocity net has ≤ 5 Heun updates; with 25 SDE updates the velocity model recovers calibration on its own. Cost: mean residualization adds ≈ 12 s of fit time on these sizes ($\sim 8\times$ vs. no residualization), so the trade is most attractive in low-step regimes.

D.2 Residualizer Variants A–E (FM side)

Five residualizer configurations were swept on a four-dataset, three-seed sub-suite (california_housing, diabetes, energy, protein-5k subsample) under VP-FM-ODE at 5 steps; see Table 10.

Within this fixed diagnostic, configurations C and E sit on the calibration–latency Pareto front. C is the headline choice (best CRPSS, $1.2\times$ faster than E); E edges C on tail coverage ($|cE|@95$ 0.010 vs. 0.011). A separate full-protein diagnostic shows E reclaiming the lead on protein (CRPSS 0.476 vs. C’s 0.449), reinforcing the case that the optimal residualizer is dataset-dependent.

Table 10: FM residualizer diagnostic. “Conf. leaves” is the LightGBM num_leaves; “ $n_{\text{est}}^{(r)}$ ” is the residualizer’s n_estimators; “ES” is early-stopping rounds in the residualizer (off means no ES).

Variant	Description	Leaves	$n_{\text{est}}^{(r)}$	ES	CRPSS $\uparrow$	q -MACE $\downarrow$	cE @95 $\downarrow$	fit s
A	current-baseline	31	100	off	0.521	0.028	0.017	1.82
B	regularised	31	100	30	0.509	0.042	0.018	2.78
C	high capacity	63	300	off	0.523	0.039	0.011	8.03
D	ES moderate	63	500	30	0.516	0.033	0.021	5.91
E	ES + high capacity	63	2000	30	0.522	0.034	0.010	9.81

D.3 Scale-Aware Residualization

The mean_scale residualizer additionally divides residuals by a cross-validated conditional-std estimate $\hat{\sigma}(x)$. On the ten-dataset mixed synthetic/real diagnostic suite with ten seeds:

Table 11: Mean vs. mean-scale residualization on FM at matched configuration. Higher CRPSS is better; lower CRPS, DSS, q -MACE are better.

Variant	CRPSS $\uparrow$	CRPS $\downarrow$	DSS $\downarrow$	q -MACE $\downarrow$
VP-FM-ODE, mean	0.519	0.652	+0.314	0.031
VP-FM-ODE, mean-scale	0.489	0.724	+0.665	0.038
VP-FM-SDE, mean	0.517	0.657	+0.380	0.040
VP-FM-SDE, mean-scale	0.475	0.750	+0.676	0.049

Scale-aware residualization is uniformly worse: +11% CRPS on the ODE path, +14% on the SDE path, with DSS more than doubling. The per- x scale estimator is itself noisy in these finite-sample regimes; dividing by it injects more variance into the velocity target than it removes. mean is therefore the only residualizer carried into the headline.

E Time Sampling and Loss Weighting

E.1 Time Sampling

Table 12: FM time-sampling sweep on fixed UCI train/test splits, 10 datasets $\times$ 3 seeds, matched residualizer-C, VP path. “uniform” is the headline choice with an endpoint anchor at $t = 1$ at probability 0.05.

Variant	CRPSS $\uparrow$	DSS $\downarrow$	q -MACE $\downarrow$	KS $p > .05$ $\uparrow$	samp s
VP-FM-ODE uniform	0.6564	−0.154	0.040	0.47	3.63
VP-FM-ODE uniform anchor16	0.6560	−0.171	0.040	0.53	3.84
VP-FM-ODE logbeta	0.6556	−0.175	0.035	0.77	3.74
VP-FM-ODE logSNR	0.6538	−0.135	0.042	0.40	3.71
score ⁺ uniform- t	0.6556	−0.210	0.038	0.50	6.95
score ⁺ log- σ t	0.6565	−0.169	0.038	0.70	6.58

For FM, log-beta and log-SNR sampling do not move CRPS by more than 0.4% relative to uniform with endpoint anchor; the smoother density shape gains +5% on KS pass rate and −13% on q -MACE but the effect is within seed variance for CRPSS. The headline keeps uniform- t for parsimony; log-beta is a viable substitute when KS uniformity is the priority. For the score model, log-sigma sampling clearly helps PIT uniformity (KS pass 0.50 $\rightarrow$ 0.70) at the cost of slightly worse DSS; earlier score-side sweeps found the EDM-default $\mathcal{N}(-1.2, 1.2^2)$ setting to be the safest dataset-agnostic choice among the tested log- σ distributions. We keep log-sigma in the SCORE⁺ headline because the KS gain is the more interpretable calibration win.

Table 13: Min-SNR- γ loss weighting on FM, fixed UCI train/test splits, 10 datasets $\times$ 3 seeds. $\gamma = 5$ collapses to uniform on most of the support; $\gamma = 1$ amplifies the late-time loss most aggressively.

Variant	CRPSS $\uparrow$	CRPS $\downarrow$	DSS $\downarrow$	q -MACE $\downarrow$	KS $p > .05 \uparrow$
VP-FM-ODE uniform- w	0.6564	4.097	-0.159	0.040	0.47
VP-FM-ODE min-SNR-5	0.6564	4.119	-0.167	0.037	0.60
VP-FM-ODE min-SNR-1	0.6516	4.223	-0.104	0.044	0.37
VP-FM-SDE uniform- w	0.6540	4.114	-0.080	0.051	0.30
VP-FM-SDE min-SNR-5	0.6538	4.138	-0.085	0.050	0.30
VP-FM-SDE min-SNR-1	0.6485	4.239	-0.046	0.057	0.23

E.2 Loss Weighting (min-SNR- γ)

Min-SNR- γ is a sharpness-versus-tail trade-off. $\gamma = 1$ hurts CRPS by 0.8–1.3% across both samplers (Wilcoxon $p \leq 0.003$ for both); $\gamma = 5$ is statistically indistinguishable from uniform weighting on CRPS, but improves PIT KS pass rate on the ODE path. We keep uniform weighting in the FM headline.

F Stochasticity (Velocity-Noise) Sweeps

F.1 Stochasticity Strength

We sweep velocity stochasticity on the VP path under linear and $\sqrt{t}$ schedules with residualizer-C, plus one residualizer-E check; results in Table 14 are averages over fixed UCI train/test splits, 10 datasets $\times$ 3 seeds at 25 SDE steps.

Table 14: Stochasticity sweep. The $\varepsilon = 0$ corner is the deterministic FM recipe (Heun ODE @ 5 steps); SDE rows use 25 steps. Sample time is omitted because the ODE and SDE rows use different benchmark protocols.

Variant	CRPSS $\uparrow$	CRPS $\downarrow$	cE @90 $\downarrow$	cE @95 $\downarrow$
$\varepsilon = 0$ headline (ODE)	0.669	—	0.024	0.017
$\varepsilon = 0.25$ linear, res-C	0.656	4.099	0.028	0.015
$\varepsilon = 0.50$ linear, res-C	0.655	4.104	0.036	0.017
$\varepsilon = 1.00$ linear, res-C	0.654	4.114	0.047	0.022
$\varepsilon = 0.50$ sqrt, res-C	0.655	4.101	0.035	0.017
$\varepsilon = 1.00$ sqrt, res-C	0.655	4.107	0.043	0.020
$\varepsilon = 1.00$ linear, res-E	0.651	4.128	0.048	0.024

In this fixed diagnostic sweep, no tested SDE setting Pareto-dominates $\varepsilon = 0$. The closest contender ($\varepsilon = 0.25$ linear) matches |cE|@95 (0.015 vs. 0.017) but is strictly worse on CRPSS and |cE|@90, and pays 25 SDE steps vs. 5 ODE steps.

F.2 Schedule Shape

The schedule $\varepsilon(t) = ct$ (linear) is the headline form. The $\sqrt{t}$ alternative was tested for parity at $\varepsilon \in \{0.5, 1.0\}$ and produces statistically indistinguishable CRPSS (Table 14). Both schedules vanish at the data endpoint, which suppresses recovered-score error in the stochastic drift (Eq. (2) and Proposition 2); neither shape helps once ε is small.

G Fixed Path Diagnostic

This fixed diagnostic is meant to isolate geometry, not to select the final path. Under matched residualizer-C, a shared 5-step ODE sampler, and fixed LightGBM settings, VP is best on CRPS, while trig and VP both reduce 95% coverage error relative to the linear path. The pattern tracks variance preservation: VP and trig satisfy $\alpha^2 + \beta^2 \approx 1$, keeping y_t at roughly unit scale across t and stabilising the tree regressor’s feature space. The tuned mechanism ablation in Table 5 supersedes this fixed ranking for performance claims: with per-dataset LightGBM tuning, linear has the best

Table 15: Flow-path diagnostic under the 5-step deterministic ODE sampler: linear vs. trigonometric vs. variance-preserving, 8 datasets $\times$ 3 seeds, matched residualizer-C. Lower is better for all columns; bold is best per column.

Path	CRPS $\downarrow$	q -MACE $\downarrow$	$ cE @95 \downarrow$	samp s
Linear	4.324	0.026	0.032	0.49
Trigonometric	4.306	0.028	0.028	0.43
Variance-preserving (VP)	4.294	0.028	0.028	0.46

aggregate CRPSS, trig is competitive and wins two datasets, and VP remains fastest. We therefore read path choice as a finite-model tradeoff: variance-preserving paths stabilise feature scale, while linear FM can have a simpler displacement-like velocity target that wins CRPS after residualization and tuning.

H Probabilistic-Family Baselines

We compare the headline Treeffuser variants against six external probabilistic regressors (Table 16). Hyperparameters for every displayed family are selected per dataset on the same fold-0 tuning split, then evaluated on folds 1–5. The CARD row uses a compact benchmark adapter with the same two-stage conditional-mean plus conditional-diffusion structure as CARD [Han et al., 2022], so conclusions involving CARD should be read as comparisons to this adapter rather than to a fully optimized CARD pipeline.

Table 16: Probabilistic baselines under the tuning/evaluation protocol. rel-CRPS is normalized by the best displayed variant on each dataset. Sample-time excludes fitting. The iBUG DSS is dominated by a single wine fold where the model assigns near-zero predictive variance to a mispredicted test point.

Model	CRPSS $\uparrow$	rel-CRPS $\downarrow$	CRPS $\downarrow$	DSS $\downarrow$	q -MACE $\downarrow$	$ cE @95 \downarrow$	samp s
Treeffuser-score ⁺ (ours)	0.681	1.131	4.057	−0.214	0.028	0.015	34.12
Treeffuser-FM-fast (ours)	0.680	1.117	4.121	− 0.218	0.034	0.024	14.41
Treeffuser-published	0.670	1.242	4.279	1.358	0.044	0.023	221.35
Quantile-regression LightGBM	0.668	1.459	4.026	0.016	0.033	0.044	0.30
Deep ensemble	0.654	1.519	4.008	0.090	0.050	0.018	0.03
iBUG (XGBoost)	0.649	1.471	4.193	1.85×10^7	0.069	0.065	2.89
CatBoost (uncertainty)	0.638	1.409	4.045	0.272	0.045	0.066	0.01
CARD-style diffusion	0.631	1.632	4.838	0.436	0.051	0.043	42.84
NGBoost (Gaussian)	0.601	1.550	4.299	1.957	0.052	0.101	0.04

Under this per-dataset tuning protocol, the Treeffuser variants lead on aggregate CRPSS, rel-CRPS, and coverage metrics. The external baselines are still important: deep ensembles have the best mean raw CRPS, though this scale-sensitive average is less informative about cross-dataset dominance than CRPSS or rel-CRPS, and the per-dataset winners include deep ensembles on diabetes and kin8nm, CARD on naval, and quantile-regression LightGBM on protein. Their aggregate rel-CRPS remains worse, indicating that no single tuned external family transfers as evenly across the benchmark suite as the Treeffuser variants.

I Extended Per-Dataset Results

Table 17 reports CRPS, $|cE|@90$, and sample-generation time on each of the ten benchmarks for the three headline Treeffuser variants, averaged over the five evaluation folds. The “best margin” column gives the gap between the winning variant and the second-best Treeffuser variant in raw CRPS. After per-dataset tuning, the margins are tighter: the main result is no longer a few dramatic wins, but a more balanced Pareto surface between published SDE sampling, score-side conditioning, and FM speed.

FM is the fastest Treeffuser sampler on 9/10 datasets and is especially valuable at scale: on protein the speedup is $19.3\times$ over PUBLISHED and $2.0\times$ over SCORE⁺; on california_housing the corresponding speedups are $31.6\times$ and $2.7\times$. On $|cE|@90$, however, SCORE⁺ leads on 6/10 datasets and FM on 3/10, which is why Table 1 separates normalized-CRPS and calibration conclusions.

Table 17: Extended per-dataset Treefuser results, ordered roughly by evaluation-fold training size. “best margin” is winner-over-second-best in raw CRPS; the bold cell in each block is the per-metric winner on that row. Numbers come from `benchmarks/results/tuning/eval/*.jsonl`.

Dataset	best margin	CRPS ↓			cE @90 ↓			sample s		
		Pub.	Score ⁺	FM	Pub.	Score ⁺	FM	Pub.	Score ⁺	FM
yacht	FM +0.4%	0.438	0.284	0.283	0.092	0.025	0.081	0.3	0.3	0.1
diabetes	Sco ⁺ +1.9%	35.682	34.025	34.660	0.079	0.029	0.038	2.4	0.5	0.2
energy	FM +5.2%	0.252	0.214	0.204	0.081	0.034	0.022	2.9	0.8	0.6
concrete	Sco ⁺ +1.5%	2.457	2.126	2.158	0.034	0.025	0.023	3.0	4.8	0.8
wine	Pub +5.9%	0.300	0.319	0.318	0.037	0.006	0.016	100.4	37.0	11.9
kin8nm	FM +4.0%	0.067	0.058	0.055	0.012	0.004	0.023	22.0	20.2	9.2
power	FM +2.4%	1.671	1.572	1.535	0.013	0.004	0.006	14.6	6.7	7.6
naval	FM +3.2%	0.000218	0.000220	0.000211	0.078	0.065	0.061	207.4	91.2	29.5
cali. housing	Pub +3.6%	0.197	0.204	0.206	0.010	0.005	0.007	605.9	52.6	19.2
protein	Pub +2.7%	1.722	1.769	1.796	0.005	0.005	0.007	1254.7	127.0	64.9

Under this evaluation protocol, the per-dataset CRPS ranking reproduces the headline pattern: FM has the best average normalized CRPS, score⁺ is the more reliable calibration row, and the published SDE remains best on wine, california housing, and protein.

J Robustness by Dataset Size

Table 18 groups the tuned evaluation artifacts by post-split training rows: small datasets have at most 1,000 rows, medium datasets have 1,001–10,000 rows, and large datasets have more than 10,000 rows. The grouping sharpens the mixed-winner story. Score⁺ and FM lead the small and medium groups on CRPSS, while the two largest datasets favor the published SDE and quantile-regression LightGBM. Thus the aggregate Treefuser result is not driven by a single data-scale regime, but neither does it erase the large-dataset inversion discussed in the limitations.

Table 18: Dataset-size robustness summary. Entries are mean CRPSS within each size group after first averaging the five evaluation folds per dataset.

Model	Small (4)	Medium (4)	Large (2)
Treefuser-published	0.709	0.670	0.591
Treefuser-score ⁺	0.735	0.678	0.578
Treefuser-FM	0.731	0.683	0.573
QReg-LightGBM	0.711	0.666	0.587
CatBoost-unc.	0.724	0.612	0.517
NGBoost	0.718	0.550	0.468
Deep ensemble	0.699	0.672	0.529
iBUG	0.700	0.637	0.571
CARD-style diffusion	0.632	0.667	0.555

The rank diagnostics in Table 4 give the same guarded conclusion. The Friedman test detects differences among the nine families, but Nemenyi post-hoc comparisons at $\alpha = 0.05$ only separate score⁺ from NGBoost and FM from NGBoost/CARD-style diffusion. They do not separate FM, score⁺, the published SDE, QReg-LightGBM, or deep ensembles from one another, so we use these ranks as robustness context rather than as a stronger significance claim.

K Conditional Tail Calibration Diagnostic

Aggregate coverage can hide where a sampler is failing. As a lightweight conditional diagnostic, we bin held-out points by the model’s predicted interquartile range (IQR) and report coverage in the highest-IQR quintile, together with IQR-MACE, the mean absolute coverage error across the five predicted-IQR bins. This is not a conditional-coverage guarantee, but it tests whether the predictive intervals behave differently on points the model itself considers uncertain. Table 19 uses the tuned evaluation artifacts and the sampler-cost sweep, averaging folds within each dataset and then averaging across the ten datasets.

The diagnostic supports a tradeoff rather than a one-sided conclusion. Deterministic FM under-covers the most uncertain quintile at 95% nominal coverage: 0.906 for VP and 0.902 for linear. Adding

Table 19: Conditional calibration proxy using predicted-IQR bins, averaged over the ten UCI datasets after fold averaging. Top-IQR Cov@ q is empirical coverage in the highest predicted-IQR quintile; IQR-MACE@ q is mean absolute coverage error across all five predicted-IQR bins. FM-SDE rows are eval-only sampler changes from the tuned FM configurations.

Row	Top-IQR Cov@90	IQR-MACE@90	Top-IQR Cov@95	IQR-MACE@95	cE @95
Published Euler-50	0.927	0.053	0.961	0.030	0.023
Score ⁺ PF-ODE-25	0.880	0.042	0.932	0.031	0.015
FM-VP ODE-5	0.854	0.051	0.906	0.040	0.024
FM-VP SDE-25	0.902	0.050	0.944	0.030	0.018
FM-linear ODE-5	0.848	0.048	0.902	0.035	0.020
FM-linear SDE-25	0.891	0.053	0.935	0.034	0.021

stochasticity lifts those numbers to 0.944 and 0.935, respectively, and improves VP’s IQR-MACE@95 from 0.040 to 0.030. However, the same SDE rows require 25 stochastic steps and, as Figure 1 shows, are not the aggregate CRPS/latency default. The published SDE and score⁺ remain competitive on this diagnostic, so the safest interpretation is that stochastic FM is a useful conditional-calibration option, not a uniformly better sampler.

L Reproducibility

The submission includes an anonymized frozen repository via `anonymous.4open.science`. The repository contains the implementation in `src/`, the benchmark harness in `benchmarks/`, the paper sources in `paper/`, and the `pixi.toml/pixi.lock` environment used for the reported runs. All sweeps in this paper are driven by YAML configs under `benchmarks/configs/`. Each result file is a JSON Lines document recording the full hyperparameter set, random seeds, the git commit hash of the model code, a hash of the Treefuser source tree, and every reported metric on a per-dataset/per-fold row. The headline tuning runs persist their selected configurations under `benchmarks/results/tuning/best_params/` and their five-fold evaluation rows under `benchmarks/results/tuning/eval/`. The configs and result artifacts that produced the tables are listed alongside in Table 20; the paper PDF is rebuilt with the paper pixi task.

Table 20: Provenance: appendix tables and the config / result file each was produced from.

Table	Config	Result file
9	<code>fm_off_vs_mean_sweep.yaml</code>	<code>fm_off_vs_mean_sweep_*.jsonl</code>
6	<code>analytic summary</code>	<code>-</code>
10	<code>residualizer_fm_sweep.yaml</code>	<code>residualizer_fm_sweep_*.jsonl</code>
11	<code>fm_mean_scale_sweep.yaml</code>	<code>fm_mean_scale_sweep_*.jsonl</code>
12	<code>fm_t_sampling_sweep.yaml</code>	<code>fm_t_sampling_sweep_*.jsonl</code>
13	<code>fm_loss_weighting_sweep.yaml</code>	<code>fm_loss_weighting_sweep_*.jsonl</code>
14	<code>sde_lower_stoch_sweep.yaml</code> , <code>stochasticity_schedule_sweep.yaml</code>	<code>sde_lower_stoch_sweep_*.jsonl</code> , <code>stochasticity_schedule_sweep_*.jsonl</code>
15	<code>ode_vs_sde_path_ablation.yaml</code>	<code>ode_vs_sde_path_ablation_*.jsonl</code>
16, 3, 17, 1	<code>tuning_manifest.yaml</code>	<code>benchmarks/results/tuning/eval/*.jsonl</code>
4, 18	<code>benchmarks/scripts/summarize_robustness.py</code>	<code>benchmarks/results/selected/robustness_summary.md</code>
5	<code>mechanism_ablation_manifest.yaml</code>	<code>benchmarks/results/mechanism_ablation/eval/*.jsonl</code>
1, 19	<code>benchmarks/scripts/run_sampler_cost_sweep.py</code> , <code>benchmarks/scripts/summarize_sampler_cost_sweep.py</code>	<code>benchmarks/results/sampler_cost_sweep/eval/sampler_cost_sweep.jsonl</code>
7	<code>benchmarks/tuning/search_spaces.py</code>	<code>-</code>

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.
- Keep the checklist subsection headings, questions/answers and guidelines below.
- Do not modify the questions and only use the provided macros for your answers.

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- 743 • Providing as much information as possible in supplemental material (appended to the
744 paper) is recommended, but including URLs to data and code is permitted.

745 **6. Experimental setting/details**

746 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
747 rameters, how they were chosen, type of optimizer) necessary to understand the results?

748 Answer: **[TODO]**

749 Justification: **[TODO]**

750 Guidelines:

- 751 • The answer [N/A] means that the paper does not include experiments.
752 • The experimental setting should be presented in the core of the paper to a level of detail
753 that is necessary to appreciate the results and make sense of them.
754 • The full details can be provided either with the code, in appendix, or as supplemental
755 material.

756 **7. Experiment statistical significance**

757 Question: Does the paper report error bars suitably and correctly defined or other appropriate
758 information about the statistical significance of the experiments?

759 Answer: **[TODO]**

760 Justification: **[TODO]**

761 Guidelines:

- 762 • The answer [N/A] means that the paper does not include experiments.
763 • The authors should answer **[Yes]** if the results are accompanied by error bars, confidence
764 intervals, or statistical significance tests, at least for the experiments that support the
765 main claims of the paper.
766 • The factors of variability that the error bars are capturing should be clearly stated (for
767 example, train/test split, initialization, random drawing of some parameter, or overall
768 run with given experimental conditions).
769 • The method for calculating the error bars should be explained (closed form formula,
770 call to a library function, bootstrap, etc.)
771 • The assumptions made should be given (e.g., Normally distributed errors).
772 • It should be clear whether the error bar is the standard deviation or the standard error
773 of the mean.
774 • It is OK to report 1-sigma error bars, but one should state it. The authors should
775 preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis
776 of Normality of errors is not verified.
777 • For asymmetric distributions, the authors should be careful not to show in tables or
778 figures symmetric error bars that would yield results that are out of range (e.g., negative
779 error rates).
780 • If error bars are reported in tables or plots, the authors should explain in the text how
781 they were calculated and reference the corresponding figures or tables in the text.

782 **8. Experiments compute resources**

783 Question: For each experiment, does the paper provide sufficient information on the com-
784 puter resources (type of compute workers, memory, time of execution) needed to reproduce
785 the experiments?

786 Answer: **[TODO]**

787 Justification: **[TODO]**

788 Guidelines:

- 789 • The answer [N/A] means that the paper does not include experiments.
790 • The paper should indicate the type of compute workers CPU or GPU, internal cluster,
791 or cloud provider, including relevant memory and storage.
792 • The paper should provide the amount of compute required for each of the individual
793 experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.